# Kosmocrates Harpoon Master Specification

v0.1 – Repository-Derived Companion to the Wonderlamp Distillation Edition

Mycelial Corpus Assimilation, Target-Bound Scraping, and Evidence-Gated Pattern Nutrition

Author: Sebastian Klemm
Synthesis: Harpoon module extracted from the Wonderlamp repository and refounded for Kosmocrates

Date: May 29, 2026

**Document status.** This is a normative implementation-oriented companion specification. It does not replace the *Kosmocrates Workbench Master Specification v0.1 – Wonderlamp Distillation Edition*. It extends it with the Harpoon subsystem found in the Wonderlamp repository. Harpoon is treated as a powerful but incomplete prototype: a cascading targeted scrape engine that must be rebuilt as a governed, evidence-bound, prompt-injection-resistant, license-aware, target-bound corpus assimilation layer for Kosmocrates.

# Contents

# Executive Summary

Harpoon is the missing corpus-assimilation organ of the Kosmocrates Workbench. In the Wonderlamp repository, Harpoon appears as a cascading targeted scrape mechanism: a seed repository or URL is analyzed, keywords are extracted from its topology and imports, external repositories are searched, cloned, parsed, reduced to observations, fed into the norm system, and used to generate the next wave of keywords. The current implementation is already structurally important, but it is not yet the final form.

The correct target form is:

> **Harpoon is a target-bound mycelial corpus assimilation layer. It grows through external keyspaces, extracts structural nutrients from code corpora, and feeds only evidence-backed, taint-aware, non-copying pattern material into Kosmocrates.**

The workpiece is the host: a repository, project, codebase, or system that is being developed. Harpoon scans that host, computes its deficiency vector, then sends hyphae through external keyspaces to retrieve relevant structural nutrients: imports, types, layer patterns, architectural motifs, tests, package idioms, common interfaces, failure modes, and norm candidates. It does not blindly copy code. It digests code into topology.

## Core transformation

```
Wonderlamp Harpoon:
  seed -> keywords -> GitHub search -> clone -> parse -> observe -> new keywords -> cascade

Kosmocrates Harpoon:
  host repo -> deficiency vector -> governed source frontier -> sandboxed acquisition
  -> topology digestion -> nutrient extraction -> quarantine -> evidence gates
  -> NormCandidate / PatternCandidate / ContextPack -> Workbench / Foundry / Crystal
```

## Central doctrine

> **External code may feed structure. External code may not directly become trusted code, trusted memory, trusted governance, or authorized action.**

## Why this matters

The Workbench Master Spec already defines a governed AI workshop. Harpoon supplies the external nutrient layer required for self-improving, corpus-aware development. Without Harpoon, the workbench learns mainly from its own runs. With Harpoon, it can grow its normic memory from the world - but only under Kosmocrates gates.

# Part I

# Repository Findings

# 1 Harpoon in the Wonderlamp Repository

## 1.1 Inspected repository surfaces

The uploaded Wonderlamp repository contains a concrete Harpoon prototype across the following surfaces:

- `crates/isls-cli/src/cmd_harpoon.rs`: CLI front-end for cascading targeted scrape.
- `crates/isls-gateway/src/discover.rs`: REST API, job state, async cascade loop, GitHub search, clone-and-analyze pipeline, keyword persistence, WebSocket events.
- `crates/isls-gateway/src/static/studio.html`: Studio commands, manual harpoon trigger, and real-time event rendering.
- `crates/isls-gateway/src/ws.rs`: Harpoon event kinds.
- `crates/isls-code-topo/src/lib.rs`: code topology signatures used during analysis.
- `crates/isls-code-topo/src/bridge.rs`: conversion from code observations into norm-learning artifacts.
- `crates/isls-norms/src/spectroscopy.rs`: keyword extraction from struct names, imports, and primary language.
- `crates/isls-norms/src/lib.rs`: norm registry, semantic observation, candidate promotion, ledger and storage interaction.

## 1.2 Current Harpoon behavior

The current Harpoon behavior can be summarized as follows.

```
Input:
  seed: local path or git URL
  depth: 1..3
  repos_per_keyword at depth 0: default 5, max 10
  stop_at_coverage: default 0.9
  domain: inferred from path/URL unless provided

Current loop:
  1. Analyze seed repository or local directory.
  2. Extract auto keywords from topology.struct_names, imports, primary language.
  3. For each depth level:
       a. Search GitHub for each keyword.
       b. Shallow-clone selected repositories.
       c. Parse files via isls-reader.
       d. Compute code topology via isls-code-topo.
       e. Convert code observations to norm artifacts.
       f. Feed artifacts to NormRegistry.observe_and_learn.
       g. Extract new keywords.
       h. Persist suggested keywords.
       i. Publish WebSocket events.
  4. Stop at max depth, max repos, empty frontier, or coverage threshold.
```

The current constants are significant:

- `HARPOON_MAX_REPOS = 200`
- `HARPOON_RPK = [5, 3, 2]`
- maximum cascade depth is clamped to `1..3`
- keyword extraction returns at most 5 keywords per analyzed repository
- network scrape workers are bounded by the mass-scrape parallelism value used elsewhere

## 1.3 Current data objects

The prototype exposes a minimal request and job shape:

```
HarpoonRequest {
  seed: String,
  depth: Option<usize>,
  repos_per_keyword: Option<usize>,
  stop_at_coverage: Option<f64>,
  domain: Option<String>,
}

HarpoonJob {
  id: String,
  status: "running" | "complete" | "stopped" | "error",
  depth_current: usize,
  depth_max: usize,
  repos_scraped: usize,
  repos_failed: usize,
  new_candidates: usize,
  new_norms: usize,
  coverage: f64,
  keywords_used: Vec<String>,
  stop_reason: Option<String>,
}
```

## 1.4 Current API and UI

The gateway registers:

```
POST /api/discover/harpoon
GET  /api/discover/harpoon/{id}/status
GET  /api/discover/suggested-keywords
POST /api/discover/accept-keyword
```

The Studio recognizes `harpoon <path_or_url>` or `harpune <path_or_url>`, starts the job, and displays WebSocket events:

```
harpoon_stage
harpoon_repo
harpoon_stage_complete
harpoon_complete
```

## 1.5 What is already correct

The prototype already contains the essential intuition:

- cascading discovery rather than one-shot scraping;
- topology-derived keywords rather than only manually curated queries;
- shallow clones and temporary directories;
- parsing before learning;
- conversion to norm-learning artifacts rather than raw code reuse;
- live operator feedback via WebSocket;
- early stopping by coverage;
- separation of curated keywords and suggested keywords.

## 1.6 What is incomplete

The prototype is not yet Kosmocrates-grade. The following weaknesses must be corrected:

- Job state is mostly in-memory; a Harpoon run is not a first-class replayable evidence object.
- External sources are not assigned formal authority and taint classes.
- License, secret, malware, and prompt-injection boundaries are not formal gates.
- Source acquisition is not yet bound to a Genesis Crystal or Capability Lock.
- Keyword frontiers are not represented as a graph with provenance, scores, or replayable transitions.

- Coverage uses a fixed coarse target count and is not host-specific.
- Current scraping is world-facing but not sufficiently host-directed.
- The system extracts norms, but does not yet model the host as a nutrient demand vector.
- Suggested keywords can influence future searches without enough governance.
- There is no robust separation between external corpus evidence and trusted pattern memory.

# Part II

# Target Concept

# 2 Harpoon as Mycelial Corpus Assimilation

## 2.1 Operational metaphor

The mycelial metaphor is technically useful if it is mapped to explicit objects.

| Metaphor | Technical object | Meaning |
|---|---|---|
| Host / workpiece | `HostProject` | Repository or target system being developed |
| Tank / substrate | `HarpoonChamber` | Isolated analysis environment around the host |
| Hypha | `HyphaEdge` | Directed expansion from one query/-source to another |
| Spore | `QuerySpore` | Search term, package name, topic, dependency, struct-derived concept |
| Nutrient | `NutrientPacket` | Extracted structural invariant, pattern, norm evidence, code topology |
| Digestion | `TopologyDigestion` | Parser + code topology + resonite extraction |
| Assimilation | `AssimilationGate` | Evidence-gated transformation into PatternCandidate or NormCandidate |
| Mycelium map | `HarpoonFrontierGraph` | Replayable graph of all queries, repos, sources, and extracted nutrients |

## 2.2 Definition

> **Harpoon is the Kosmocrates subsystem that binds a host repository to an expanding, governed source frontier; traverses external keyspaces; extracts structural nutrients; and feeds only evidence-backed, taint-aware candidate knowledge into the Workbench.**

## 2.3 The host-first inversion

The original prototype begins with a seed and grows outward. The Kosmocrates version begins with a host deficiency model.

```
Prototype:
  seed -> keywords -> repos -> keywords -> repos

Target:
  host -> deficiency vector -> query spores -> source frontier
       -> topology nutrients -> candidate memory -> workbench context
```

**Invariant HAR-001: Host-bound harvesting.** Every Harpoon run **MUST** bind to exactly one primary `HostProject`. A general corpus-growth mode **MAY** exist, but it must still declare a virtual host profile and a purpose-bound acquisition policy.

**Invariant HAR-002: Nutrient, not code.** External corpus content **MUST** be reduced to structural nutrients before it may influence Workbench generation. Direct code reuse **MUST NOT** occur unless a separate license-aware, human-approved import workflow authorizes it.

## 2.4 Harpoon and Kosmocrates

Harpoon is not a separate intelligence engine. It is a Kosmocrates-bound acquisition and digestion layer.

```
HostProject
-> Harpoon Chamber
-> Source Frontier
-> Corpus Acquisition
-> Topology Digestion
-> Nutrient Extraction
-> Quarantine
-> Kosmocrates Gates
-> Pattern / Norm / Context objects
-> Workbench / Foundry / Agenda
```

# Part III

# Architecture

# 3 Harpoon Layer Stack

## 3.1 Layer overview

| Layer | Responsibility |
|---|---|
| H0 Host Binding | Select workpiece, compute digest, scan topology, bind Genesis and policy. |
| H1 Deficiency Model | Compute missing capabilities, weak layers, absent tests, low coverage, architectural gaps. |
| H2 Query Spore Factory | Generate search spores from host topology, imports, Norm gaps, ETV retrieval, and user goal. |
| H3 Frontier Graph | Maintain query/source/repository graph with scores, parentage, depth, budgets, and taint. |
| H4 Source Acquisition | Search, fetch, clone, or read sources under capability, sandbox, rate, size, and allowlist constraints. |
| H5 Topology Digestion | Parse source, compute code topology, extract resonites, imports, layer motifs, signatures. |
| H6 Nutrient Extraction | Produce structured NutrientPackets: norms, archetype evidence, test motifs, interface motifs, dependencies. |
| H7 Quarantine and Gates | Assign taint, license signals, secret signals, injection signals, and validation status. |
| H8 Assimilation | Promote only gate-passed nutrients into PatternCandidate, NormCandidate, ContextPack, or AgendaItem. |
| H9 Feedback | Update host deficiency vector, query frontier, ETV retrieval, and Workbench task planning. |

## 3.2 Canonical pipeline

```
HarpoonRun
  Input: HostProject + OperatorGoal + HarpoonPolicy

  1. Bind host: canonical digest, workspace topology, current Kosmocrates context.
  2. Compute deficiency vector: missing norms, weak tests, missing layers, broken interfaces.
  3. Generate spores: keywords, package queries, import-derived concepts, archetype gaps.
  4. Expand frontier: search configured source adapters under policy.
  5. Acquire sources: sandboxed shallow clone/fetch, size/time/network bounds.
  6. Digest sources: parse, topology, resonites, language/layer signatures.
  7. Extract nutrients: structural invariants, examples, anti-examples, candidate norms.
  8. Quarantine: license, taint, injection, secret, malicious-content checks.
  9. Score relevance: host gap alignment, topology similarity, novelty, evidence strength.
 10. Assimilate: produce candidate memory, not trusted memory, unless gates pass.
 11. Report: evidence bundle, frontier graph, nutrient table, recommendations.
```

# 4 Core Data Model

**Type HostProject.**

```
pub struct HostProject {
    pub id: Hash256,
    pub root_path: PathBuf,
    pub repo_url: Option<String>,
    pub branch: Option<String>,
    pub workspace_digest: Hash256,
    pub language_profile: LanguageProfile,
    pub topology_signature: CodeTopologySignature,
    pub norm_coverage: NormCoverageReport,
    pub genesis_crystal_id: CrystalId,
}
```

**Type HarpoonPolicy.**

```
pub struct HarpoonPolicy {
    pub max_depth: u8,
    pub max_sources: usize,
    pub max_sources_per_spore: usize,
    pub max_bytes_per_source: u64,
    pub max_total_bytes: u64,
    pub max_runtime_seconds: u64,
    pub allowed_source_adapters: Vec<SourceAdapterId>,
    pub allowed_network_domains: Vec<String>,
    pub allow_private_sources: bool,
    pub license_policy: LicensePolicy,
    pub secret_policy: SecretPolicy,
    pub taint_policy: TaintPolicy,
    pub assimilation_policy: AssimilationPolicy,
    pub require_operator_confirmation: bool,
}
```

**Type DeficiencyVector.**

```
pub struct DeficiencyVector {
    pub missing_norms: Vec<NormClass>,
    pub weak_layers: Vec<LayerWeakness>,
    pub absent_tests: Vec<TestGap>,
    pub interface_gaps: Vec<InterfaceGap>,
    pub dependency_unknowns: Vec<DependencyUnknown>,
    pub security_gaps: Vec<SecurityGap>,
    pub documentation_gaps: Vec<DocumentationGap>,
    pub priority_scores: BTreeMap<GapId, f64>,
}
```

**Type QuerySpore.**

```
pub struct QuerySpore {
    pub id: Hash256,
    pub text: String,
    pub source: SporeSource,
    pub parent: Option<Hash256>,
    pub depth: u8,
    pub target_gap: Option<GapId>,
    pub expected_nutrient: NutrientKind,
    pub score: f64,
    pub taint: TaintKind,
}
```

```
pub enum SporeSource {
    HostStructName,
    HostImport,
    NormGap,
    ArchetypeGap,
    UserGoal,
    ETVRetrieval,
    PriorNutrient,
    ManualSeed,
}
```

**Type SourceCandidate.**

```
pub struct SourceCandidate {
    pub id: Hash256,
    pub adapter: SourceAdapterId,
    pub locator: SourceLocator,
    pub display_name: String,
    pub parent_spore: Hash256,
    pub rank: usize,
    pub external_score: Option<f64>,
    pub license_hint: Option<String>,
    pub authority: AuthorityClass,
    pub taint: TaintKind,
}
```

**Type NutrientPacket.**

```
pub struct NutrientPacket {
    pub id: Hash256,
    pub source_id: Hash256,
    pub host_id: Hash256,
    pub extracted_at: Timestamp,
    pub kind: NutrientKind,
    pub topology_signature: CodeTopologySignature,
    pub resonites: Vec<Resonite>,
    pub norm_evidence: Vec<NormEvidenceFragment>,
    pub archetype_evidence: Vec<ArchetypeEvidenceFragment>,
    pub test_motifs: Vec<TestMotif>,
    pub interface_motifs: Vec<InterfaceMotif>,
    pub anti_patterns: Vec<AntiPatternSignal>,
    pub license_signal: LicenseSignal,
    pub secret_signal: SecretSignal,
    pub injection_signal: InjectionSignal,
    pub relevance_score: f64,
    pub novelty_score: f64,
    pub taint: TaintKind,
}
```

**Type HarpoonRun.**

```
pub struct HarpoonRun {
    pub id: Hash256,
    pub host: HostProject,
    pub policy: HarpoonPolicy,
    pub request: HarpoonRequest,
    pub frontier_graph: HarpoonFrontierGraph,
    pub acquired_sources: Vec<SourceCandidate>,
    pub nutrients: Vec<NutrientPacket>,
    pub gate_reports: Vec<HarpoonGateReport>,
    pub evidence_bundle: EvidenceBundleId,
    pub status: HarpoonRunStatus,
    pub stop_reason: Option<String>,
}
```

# 5 Frontier Graph and Hyphal Traversal

## 5.1 Graph model

A Harpoon run is not a list of searches. It is a graph.

```
Node types:
  HostNode
  QuerySporeNode
  SourceCandidateNode
  SourceSnapshotNode
  NutrientPacketNode
  CandidateMemoryNode
  GateReportNode

Edge types:
  generated_spore
  searched_source
  acquired_snapshot
  digested_into
  extracted_nutrient
  gated_as
  assimilated_as
  rejected_as
```

**Invariant HAR-003: Frontier replayability.** Every edge in the frontier graph **MUST** record its parent node, operator id, input digest, output digest, policy digest, and timestamp. A Harpoon report without a replayable frontier graph is non-conformant.

## 5.2 Hyphal traversal algorithm

**Algorithm HAR-TRAVERSE-001.**

```
Input: HostProject H, DeficiencyVector D, HarpoonPolicy P
Output: HarpoonRun R

frontier = seed_spores(H, D, P)
for depth in 0..P.max_depth:
    spores = select_frontier_spores(frontier, depth, P)
    for spore in spores:
        candidates = search_sources(spore, P)
        for source in ranked(candidates):
            if budget_exhausted(P): break
            if not source_gate(source, P): continue
            snapshot = acquire_source(source, P)
            digest = digest_source(snapshot, H, P)
            nutrients = extract_nutrients(digest, H, D)
            for nutrient in nutrients:
                report = gate_nutrient(nutrient, P)
                record(report)
                if report.decision == AssimilateCandidate:
                    assimilate_candidate(nutrient, H)
                    frontier.add_spores(derive_spores(nutrient, D))
                elif report.decision == Quarantine:
                    agenda.guard(nutrient)
```

```
                else:
                    record_rejection(nutrient)
    D = recompute_deficiency(H)
    if stop_condition(D, frontier, P): break
return evidence_bound_run(frontier)
```

## 5.3 Spore selection

Spore selection **MUST** combine:

- host deficiency priority;
- query novelty;
- expected nutrient class;
- depth budget;
- past source quality;
- risk and taint;
- operator goal alignment;
- ETV retrieval score.

# 6 Source Adapters

## 6.1 Adapter contract

A source adapter is a bounded, governed acquisition interface. GitHub is only one adapter.

```
pub trait HarpoonSourceAdapter {
    fn id(&self) -> SourceAdapterId;
    fn search(&self, spore: &QuerySpore, policy: &HarpoonPolicy)
        -> Result<Vec<SourceCandidate>, AdapterError>;
    fn acquire(&self, candidate: &SourceCandidate, policy: &HarpoonPolicy)
        -> Result<SourceSnapshot, AdapterError>;
    fn authority_class(&self) -> AuthorityClass;
    fn default_taint(&self) -> TaintKind;
}
```

## 6.2 Initial adapter set

| Adapter | Priority | Purpose |
|---|---|---|
| LocalRepoAdapter | Required | Analyze host and local repositories. |
| GitRepoAdapter | Required | Shallow clone explicit git URLs. |
| GitHubSearchAdapter | Required if network enabled | Search public repositories by query spores. |
| DependencyManifestAdapter | Strongly recommended | Expand from Cargo.toml, package.json, pyproject, go.mod, etc. |
| DocsAdapter | Optional | Ingest public documentation as tainted evidence. |
| IssueCorpusAdapter | Optional | Extract failure motifs and edge cases from public issues. |
| PackageRegistryAdapter | Optional | Search crates/npm/pypi/go package metadata. |

**Requirement SRC-001: Explicit network capability.** No network source adapter may run without a `NetworkAcquireCapability`. The capability **MUST** bind allowed domains, max requests, max bytes, max runtime, and allowed adapters.

**Requirement SRC-002: Public does not mean trusted.** Public code, public documentation, public package metadata, and public issues **MUST** be treated as external tainted evidence unless explicitly promoted by gates.

# Part IV

# Security, Taint, and Assimilation

# 7 Authority and Taint Model

## 7.1 Authority classes

```
A0: System / Genesis / Constitutional invariants
A1: Kosmocrates Governance / RuleAtoms
A2: Operator-approved Workbench TaskSpec
A3: Trusted local workspace state
A4: Tool output from controlled sandbox
A5: Kosmocrates Crystal memory
A6: External corpus source
A7: Unknown / hostile source
```

**Invariant AUTH-001: No authority ascent through corpus.** No object derived from A6 or A7 may become an instruction, operator, capability, trusted memory object, governance mutation, or direct code emission without an explicit evidence-bound transformation gate.

## 7.2 Taint propagation

All external source-derived nutrients carry taint.

```
ExternalRepoSource -> SourceSnapshot(TaintedExternal)
SourceSnapshot -> NutrientPacket(TaintedExternal)
NutrientPacket -> CandidateMemory(TaintedExternal | Quarantined)
CandidateMemory -> TrustedPattern only after promotion gates
```

## 7.3 Prompt injection as corpus topology attack

External repositories may contain natural-language text, comments, tests, README instructions, or code strings that attempt to manipulate the agent. The Harpoon layer treats this as an attempted authority ascent.

**Gate INJECT-001: Instruction/Data separation.** External source content may inform extraction and evidence. It **MUST NOT** be interpreted as an instruction to the Workbench, Oracle, Operator, ToolExecutor, or Governance layer.

**Gate INJECT-002: Corpus instruction quarantine.** Any detected instruction-like segment in external content **SHOULD** be recorded as `InjectionSignal`. It may be used for defensive learning but **MUST NOT** be placed in an executable prompt except as quoted, attributed, tainted evidence.

# 8 License, Secrets, and Source Hygiene

## 8.1 License boundary

Harpoon is designed to extract topology, not copy code. This is both an architectural and legal hygiene boundary.

**Gate LICENSE-001: No raw code assimilation.** Raw external source text **MUST NOT** be committed into trusted pattern memory unless a separate license gate, attribution gate, operator approval, and trace record authorize it. The default assimilation unit is structural topology, not source text.

**Gate LICENSE-002: License signal required.** Every `SourceSnapshot` **MUST** carry a `LicenseSignal`. Unknown license status **MUST** downgrade assimilation to structural-only evidence.

## 8.2 Secret boundary

External repositories may contain accidental secrets. Harpoon **MUST** not transform discovered secrets into model prompts, memory, or reports beyond redacted signals.

**Gate SECRET-001: Secret redaction.** If a secret-like token is detected, the token **MUST** be redacted before logging, prompt construction, or memory storage. The event may persist as a redacted `SecretSignal`.

# 9 Capability Locks

## 9.1 Required capabilities

Harpoon requires specific capabilities:

- `HostScanCapability`
- `NetworkSearchCapability`
- `SourceAcquireCapability`
- `SandboxParseCapability`
- `NutrientExtractCapability`
- `CandidateAssimilateCapability`
- `ReportEmitCapability`

## 9.2 Capability lock object

```
pub struct HarpoonCapabilityLock {
    pub id: Hash256,
    pub capability: HarpoonCapability,
    pub host_id: Hash256,
    pub policy_digest: Hash256,
    pub genesis_crystal_id: CrystalId,
    pub required_authority: AuthorityClass,
    pub forbidden_taints: Vec<TaintKind>,
    pub required_gate_proofs: Vec<GateProofId>,
    pub max_uses: Option<u32>,
    pub expires_at: Option<Timestamp>,
}
```

**Requirement CAP-001: No acquisition without lock.** Every source search, clone, fetch, parse, extract, and assimilation operation **MUST** be backed by a capability lock bound to the current Genesis Crystal and Harpoon policy.

# Part V

# Digestion and Nutrient Extraction

# 10 Topology Digestion

## 10.1 Source parsing

Source parsing **MUST** produce language-normalized observations. The initial implementation may reuse the Wonderlamp reader model for Rust, TypeScript, Python, Go, SQL, and JavaScript. Later implementations may use tree-sitter or dedicated parsers.

## 10.2 Code topology signature

A digested source produces a `CodeTopologySignature`:

```
pub struct CodeTopologySignature {
    pub node_count: usize,
    pub edge_count: usize,
    pub function_signatures: Vec<String>,
    pub type_names: Vec<String>,
    pub imports: Vec<String>,
    pub layers: Vec<LayerKind>,
    pub language_breakdown: BTreeMap<String, usize>,
    pub primary_language: String,
    pub connectivity: f64,
    pub structural_hash: Hash256,
}
```

## 10.3 Resonite extraction

The nutrient extractor **MUST** extract resonites:

```
Fn(name, arity)
Type(name, kind)
Import(path)
Layer(depth, artifact)
Relation(source, target, kind)
Test(name, target)
Interface(provider, consumer, contract)
Config(key, scope)
```

# 11 Nutrient Taxonomy

## 11.1 Nutrient kinds

| Kind | Meaning |
|---|---|
| NormEvidence | Evidence that a known or candidate norm appears in the source. |
| ArchetypeEvidence | Evidence for recurring multi-norm configurations. |
| InterfaceMotif | Reusable provider-consumer, route-service, service-query pattern. |
| TestMotif | Common test pattern, fixture shape, or assertion strategy. |
| DependencyMotif | Dependency usage pattern, import cluster, package convention. |
| LayerMotif | Architectural layer arrangement or separation-of-concerns evidence. |
| AntiPatternSignal | Evidence of unstable, risky, or rejected structures. |
| QuerySporeSeed | New safe search term or search direction. |

## 11.2 Nutrient scoring

Each nutrient receives scores:

```
relevance_score = alignment_with_host_gap
novelty_score   = distance_from_existing_pattern_memory
evidence_score  = source_count + independent_confirmation + parse_quality
risk_score      = taint + license_unknown + injection_signal + secret_signal
utility_score   = relevance * evidence * novelty * (1 - risk)
```

**Requirement NUT-001: Scored assimilation.** No nutrient may be assimilated into candidate memory without relevance, novelty, evidence, and risk scoring.

# 12 Assimilation Paths

## 12.1 Four destinations

A nutrient may end in exactly one of four states:
1. **Rejected**: invalid, irrelevant, unsafe, duplicate, or policy-disallowed.
2. **Quarantined**: potentially useful but tainted, risky, or insufficiently evidenced.
3. **Candidate**: usable as external evidence for a pattern/norm hypothesis.
4. **Trusted Crystal**: only after independent gates, replay, and promotion.

**Gate ASSIM-001: Candidate before crystal.** External nutrients **MUST** enter candidate memory before any possible crystal promotion. Direct corpus-to-trusted-crystal promotion is forbidden.

**Gate ASSIM-002: Independent confirmation.** A corpus-derived norm or archetype **SHOULD** require independent confirmation across multiple sources, or a controlled host-side Foundry proof, before promotion.

# Part VI

# Host-Directed Harpoon

# 13 Deficiency Vector

## 13.1 Purpose

The deficiency vector turns Harpoon from a crawler into a targeted extractor. The host tells Harpoon what it lacks.

## 13.2 Deficiency sources

A host deficiency vector may be computed from:

- failing tests;
- missing tests;
- missing layers in an expected archetype;
- unknown dependency usage;
- low norm coverage;
- low MCI / structural consistency;
- repeated LLM repair loops;
- ETV agenda items;
- user goal and TaskSpec constraints;
- security/compliance gaps.

## 13.3 Deficiency-driven search

```
Example:
  Host: Rust Axum API project
  Gap: Search endpoint + date-range filtering + tests
  Spores:
    rust axum search endpoint
    rust sqlx dynamic query filters
    rust axum query params date range
    rust integration test axum sqlx
  Nutrients:
    InterfaceMotif(route -> service -> query)
    TestMotif(date range search)
    DependencyMotif(chrono/sqlx binding)
```

# 14 Workbench Integration

## 14.1 ContextPack production

Harpoon does not patch the host by itself. It produces `HarpoonContextPack`s for the Workbench.

```
pub struct HarpoonContextPack {
    pub host_id: Hash256,
    pub task_id: TaskId,
    pub deficiency_vector_id: Hash256,
    pub selected_nutrients: Vec<NutrientPacketId>,
    pub pattern_candidates: Vec<PatternCandidateId>,
    pub norm_candidates: Vec<NormCandidateId>,
    pub warnings: Vec<HarpoonWarning>,
    pub evidence_bundle: EvidenceBundleId,
}
```

## 14.2 Workbench flow

```
User Task
-> Workspace scan
-> Gap detected
-> Harpoon run requested
-> HarpoonContextPack returned
-> Workbench builds constrained prompt/ArtifactIR
-> Oracle fills bounded slots
-> Foundry validates
-> Successful result may become host-side proof for nutrient promotion
```

**Requirement WB-001: Harpoon cannot mutate host.** Harpoon **MUST NOT** directly write host files. It may produce context, candidates, reports, and Workbench suggestions. File mutation remains under Workbench ActionCandidate and CapabilityLock governance.

# Part VII

# API, CLI, and Runtime

# 15 API Contract

## 15.1 Required endpoints

The new API **SHOULD** move from `/api/discover/harpoon` into an explicit Harpoon namespace while preserving compatibility aliases.

```
POST /api/harpoon/runs
GET  /api/harpoon/runs/{id}
POST /api/harpoon/runs/{id}/abort
GET  /api/harpoon/runs/{id}/frontier
GET  /api/harpoon/runs/{id}/nutrients
POST /api/harpoon/runs/{id}/assimilate
GET  /api/harpoon/runs/{id}/report
POST /api/harpoon/query/preview
GET  /api/harpoon/policies
POST /api/harpoon/policies
```

## 15.2 Run request

```
POST /api/harpoon/runs
{
  "host_path": "./repo",
  "goal": "improve search endpoint implementation",
  "policy": "local_safe_default",
  "max_depth": 3,
  "max_sources": 100,
  "source_adapters": ["local", "github_search", "git"],
  "operator_confirmation": true
}
```

## 15.3 Run response

```
{
  "ok": true,
  "run_id": "harpoon:<sha256>",
  "host_id": "sha256:...",
  "policy_digest": "sha256:...",
  "status": "running",
  "events": "/api/events"
}
```

# 16 WebSocket Events

## 16.1 Required event types

```
harpoon:run_started
harpoon:host_scanned
harpoon:deficiency_vector
harpoon:spore_generated
harpoon:source_found
harpoon:source_acquired
harpoon:source_rejected
harpoon:nutrient_extracted
harpoon:nutrient_quarantined
harpoon:nutrient_assimilated
harpoon:stage_complete
harpoon:run_complete
harpoon:run_error
```

## 16.2 Event payload discipline

Every event **MUST** include:

```
run_id
host_id
event_index
parent_event_digest
payload_digest
timestamp
```

# 17 CLI Contract

## 17.1 Commands

```
kosmo harpoon scan --host ./repo
kosmo harpoon run --host ./repo --goal "add search" --policy local-safe
kosmo harpoon status <run-id>
kosmo harpoon frontier <run-id>
kosmo harpoon nutrients <run-id>
kosmo harpoon report <run-id> --format md|json|html
kosmo harpoon assimilate <run-id> --candidate <id>
kosmo harpoon abort <run-id>
```

## 17.2 Compatibility command

The legacy command form may be supported:

```
kosmo harpoon --seed <path_or_url> --depth 3 --repos-per-keyword 5
```

But compatibility mode **SHOULD** internally construct a virtual `HostProject` and `HarpoonPolicy`; it must not bypass the new evidence model.

# Part VIII

# Implementation Plan

# 18 Module Map

## 18.1 Recommended crates or modules

For Kosmocrates, Harpoon may be implemented as a module family rather than one monolithic crate.

```
crates/
  kosmo-harpoon-types/       # data structures, policies, reports
  kosmo-harpoon-core/        # run engine, frontier graph, scheduling
  kosmo-harpoon-adapters/    # local/git/github/package/docs adapters
  kosmo-harpoon-digest/      # parsers, topology digestion, resonites
  kosmo-harpoon-gates/       # taint/license/secret/injection/assimilation gates
  kosmo-harpoon-api/         # REST/SSE/WS handlers
  kosmo-harpoon-cli/         # command-line front-end
```

If the Kosmocrates workspace prefers fewer crates, the same layout may be implemented as modules under `pse-workbench` or `kosmocrates-workbench`.

# 19 Phased Implementation

## 19.1 Phase H0: Extract current prototype safely

- Port request/job/status structures.
- Port seed analysis, keyword extraction, GitHub search, shallow clone, parse, topology, and suggested keyword behavior.
- Wrap all network and clone operations in policy objects.
- Preserve legacy CLI and API compatibility where practical.

## 19.2 Phase H1: Evidence-bound run model

- Add `HarpoonRun`, `HarpoonFrontierGraph`, `EvidenceBundle`.
- Persist run state append-only.
- Add replayable event digests.
- Add reports.

## 19.3 Phase H2: Authority and taint

- Add source authority classes.
- Add taint propagation.
- Add external-content instruction quarantine.
- Add secret and license signals.

## 19.4 Phase H3: Host deficiency vector

- Bind every run to a HostProject.
- Compute missing norms, missing tests, weak layers, and interface gaps.
- Generate spores from the deficiency vector.

## 19.5 Phase H4: Nutrient packets and assimilation gates

- Implement `NutrientPacket` extraction.
- Add assimilation outcomes: reject, quarantine, candidate, trusted.
- Connect candidates to Normic Pattern Memory and Epistemic Agenda.

## 19.6 Phase H5: Workbench context integration

- Emit `HarpoonContextPacks` for specific tasks.
- Feed selected nutrients into TaskSpec and ArtifactIR construction.
- Use Foundry success as promotion evidence.

## 19.7 Phase H6: Advanced frontier intelligence

- Add ETV-guided spore generation.

- Add non-dominated frontier selection.
- Add source quality memory.
- Add exploration/coverage telemetry.

# 20 Acceptance Tests

## 20.1 Core tests

| Test ID | Requirement |
|---|---|
| HT-001 | Local host scan produces HostProject with stable digest. |
| HT-002 | Seed repository produces bounded QuerySpores. |
| HT-003 | Network acquisition fails closed without capability. |
| HT-004 | Source clone respects max bytes, timeout, and temp cleanup. |
| HT-005 | External source content is tainted by default. |
| HT-006 | Instruction-like external text is quarantined and cannot become ToolAction. |
| HT-007 | NutrientPacket contains source id, host id, topology signature, taint, and evidence. |
| HT-008 | Unknown license prevents raw code assimilation. |
| HT-009 | Secret-like content is redacted before logging. |
| HT-010 | FrontierGraph is replayable and content-addressed. |
| HT-011 | HarpoonRun persists after process restart. |
| HT-012 | Legacy harpoon seed mode maps to a policy-bound run. |
| HT-013 | Candidate assimilation never creates trusted crystal directly. |
| HT-014 | Workbench receives HarpoonContextPack but Harpoon cannot mutate host files. |
| HT-015 | Foundry success can be attached as promotion evidence. |

## 20.2 Adversarial tests

- Repository README contains `ignore previous instructions`; it is detected as injection signal and never treated as instruction.
- Repository contains fake system prompt in comments; it remains quoted tainted evidence.
- Repository contains suspected API key; report redacts it and emits a secret signal.
- Repository without license may yield structural topology but no raw code reuse.
- GitHub search adapter returns duplicate repositories; frontier deduplicates by canonical source digest.
- Malformed repository fails parsing but produces an evidence-bound rejection report.

# 21 Definition of Done

A conformant Harpoon implementation is complete when:

1. every run is bound to a HostProject, policy digest, Genesis Crystal, and evidence bundle;
2. all external sources are tainted by default;
3. all acquisition and assimilation operations require capability locks;
4. source frontier transitions are represented in a replayable graph;
5. nutrients are extracted as structural objects, not raw copied code;
6. license, secret, prompt-injection, and source hygiene gates exist;
7. nutrients can be rejected, quarantined, assimilated as candidates, or promoted only through gates;
8. Harpoon cannot directly mutate host files;
9. Workbench can consume HarpoonContextPacks;
10. Foundry success can feed promotion evidence;
11. all required API, CLI, and event surfaces are implemented;
12. all acceptance tests pass;
13. the documentation states that Harpoon is a corpus-assimilation layer, not a code-copying scraper.

# Part IX

# Appendices

# 22 Current-to-Target Mapping

| Current Wonderlamp element | Current function | Kosmocrates target |
|---|---|---|
| `cmd_harpoon.rs` | CLI cascading scrape | `kosmo harpoon` commands with policy and evidence |
| `discover_harpoon` | Start gateway job | `POST /api/harpoon/runs` |
| `HarpoonJob` | In-memory status | Persistent `HarpoonRun` |
| `HARPOON_RPK` | Fixed depth schedule | Policy-driven spore selection |
| `suggested_keywords.txt` | Keyword frontier storage | FrontierGraph + operator-curated spores |
| `github_search_repos` | GitHub source search | SourceAdapter interface |
| `clone_and_analyze_sync` | Clone/parse/topology/norm feed | Sandboxed acquisition + digestion + nutrient extraction |
| `extract_keywords_from_analysis` | Struct/import/language keywords | QuerySpore factory |
| `NormRegistry.observe_and_learn` | Norm candidate update | Candidate assimilation under gates |
| `harpoon_* WS events` | UI progress | Evidence-bound run event stream |

# 23 Minimal Viable Harpoon

## 23.1 MVH requirements

A minimal viable Harpoon implementation requires:

- local host scan;
- seed-based query spores;
- GitHub or git source adapter behind capability lock;
- shallow clone to temp sandbox;
- parser/topology digestion;
- NutrientPacket output;
- taint propagation;
- no direct host mutation;
- report generation;
- Workbench ContextPack export.

## 23.2 First implementation task

The first coding-agent task should be:

> Implement persistent `HarpoonRun` and `HarpoonFrontierGraph` around the existing `cmd_harpoon.rs` and `discover.rs` behavior, without changing scraping semantics, then add taint and evidence fields to every source and nutrient object.

# 24 Glossary

| Term | Definition |
|---|---|
| Harpoon | Target-bound corpus assimilation layer. |
| HostProject | The repository or workpiece being improved. |
| Hypha | Frontier edge from one information node to another. |
| QuerySpore | Search query generated from host gaps or prior nutrients. |
| SourceCandidate | External repository, package, document, or source to inspect. |
| SourceSnapshot | Sandboxed acquired source state. |
| TopologyDigestion | Parser/topology/resonite extraction from source. |
| NutrientPacket | Structured extracted invariant or evidence fragment. |
| Assimilation | Gate-controlled movement from nutrient to candidate memory. |
| DeficiencyVector | Host-specific vector of missing/weak capabilities. |
| FrontierGraph | Replayable graph of Harpoon traversal. |

# 25 Closing Statement

Harpoon is not merely a scraper. The prototype already shows a more important shape: a recursive, topology-driven, keyword-evolving extraction organ. Kosmocrates turns that organ into a governed subsystem.

The final design is therefore:

> **Harpoon grows through the world's code keyspaces, but Kosmocrates decides what can nourish the host.**